Aghalya Narayanan aghalya2
Charan Williams charanw2
Ganesh Saranga saranga2
Jingqia Liu jl126
Keegan Miller kmill34
Nikhil Ajit najit2
Noah Denefe ndenefe2
Zian Xiang zianx2

4th Credit Hour Assignment: LLM Agents

November 9th, 2025

Performance

Subset	Resolved Count	Resolved ID
verified-40	8	"django__django-11095", "django__django-11532", "django__django-11880", "django__django-13363", "django__django-15315", "django__django-15368", "django__django-15987", "django__django-16661"
verified-45-extra	optional	optional
verified-50-extra	optional	optional

Design

Read and Write Tool

- Why do you think it is useful?

  The Read and Write tool gives the LLM agent more flexible “hands” to interact with code and files. Before this tool, the LLM could only modify code indirectly through shell commands like “sed” or “awk”, which required the LLM to rely solely on BASH scripting, which can be inefficient. The Read and Write tool instead uses a structured JSON format that is easier to interpret and less prone to errors. In addition, it could help

prevent potentially dangerous command line operations by safely handing inputs (for example, through shlex.quote() in Python.)

- Are there alternatives, and why are they not chosen?

  Yes. For example, alternatives such as SandBoxed Editors can also be used to debug or edit files within a virtual environment. However, it requires significantly more memory to maintain the sandbox. SWE-Agent utilizes Docker containers for only specific parts of the workflow execution, keeping the process efficient. Additionally, a virtual environment may not fully replicate the real system environment and is temporary, making synchronization more challenging. In contrast, the Read and Write tool, integrated within the SWE-agent, allows for more direct, efficient, and synchronized interaction with project files.

- How did you implement it?

  Implementation of the write feature was based on the existing read feature provided with the project. It takes a string path of the text file to be written and string content to be written, and writes the content to a UTF-8 text file. It includes guards for missing paths and content, and attempts to write the content in the sandbox environment. It creates a shell command using shlex.quote(), which ensures the file path is safe to pass to the shell. It also ensures the parent directory exists using mkdir and then writes the file using cat. It includes a fallback to write using the host filesystem, similar to the provided read feature, and throws FileNotFound, IsADirectory, and Permission Errors, with a generic Exception fallback. The host filesystem write function should not be used, but it was included for now, as it was provided with the read function. The logic is similar, except that it is implemented in Python, making it cross-platform compatible. No significant challenges were encountered. A current limitation is that only string content can be written. The function could be improved to write binary.

- Was it actually useful?

  In the generated patches so far, an example of the agent using the read_file tool can be found in django_django-11880.traj.json. From this, we can see that the tool allowed the agent to read django/forms/fields.py and confirm that __deepcopy__() did not copy the error_messages. It was then able to insert a line to fix it. This tool enabled the agent to view the source code, verify the problem, and determine where a patch was needed.

  For the patches generated at this point of the project, a search was done for usage of the write tool. Usage was not found. However, the model could likely use this tool in combination with further tools we plan to develop. Presently, the model utilizes a standard, unified git diff command to apply its patches.

Below are the returned “thoughts” for instance django_django-11880:

THOUGHT: The PR description states that the `__deepcopy__` method for form fields in Django does not deep copy the `error_messages` dictionary. This leads to all copies of a field sharing the same `error_messages` dictionary, causing incorrect behavior when error messages are modified dynamically.

My first step is to locate the relevant file mentioned in the PR description: `django/forms/fields.py`. I'll start by listing the contents of the current directory to understand the project structure.

```bash
ls -R
```

THOUGHT: The PR description states that the `__deepcopy__` method for form fields in Django does not deep copy the `error_messages` dictionary. This leads to all copies of a field sharing the same `error_messages` dictionary, causing incorrect behavior when error messages are modified dynamically.

My first step is to locate the relevant file mentioned in the PR description: `django/forms/fields.py`. I'll start by listing the contents of the current directory to understand the project structure.

```bash
ls -R
```

THOUGHT: The `ls -R` command produced too much output, but I can see the `django/forms/fields.py` path in the output. Now I need to read the content of this file to understand the current implementation of `__deepcopy__` and identify where the `error_messages` dictionary is handled. I'll use `read_file` to view the file.

```tool
{"name": "read_file", "args": {"path": "django/forms/fields.py"}}
```

THOUGHT: I've read the file `django/forms/fields.py`. I need to find the `__deepcopy__` method within the `Field` class. The PR description mentions line 200, which is around where the `__init__` method is defined. I'll search for `__deepcopy__` in the file. If it's not explicitly defined, then the default shallow copy behavior of Python's `copy.deepcopy` would apply, which means the `error_messages` dictionary would indeed be shallow copied.

Looking at the `__init__` method, I see this:

```python
        messages = {}
        for c in reversed(self.__class__.__mro__):
            messages.update(getattr(c, 'default_error_messages', {}))
        messages.update(error_messages or {})
        self.error_mess
```

It seems the `error_messages` are constructed by updating a dictionary. If `__deepcopy__` is not explicitly defined, then `self.error_messages` (which is likely `self.error_messages` and not `self.error_mess` as truncated in the output) would be shallow copied.

I'll search for `def __deepcopy__` in the file to confirm if it exists.

```bash
grep -n "def __deepcopy__" django/forms/fields.py
```

THOUGHT: Okay, `grep` found three `__deepcopy__` methods. The PR description specifically points to `django/forms/fields.py#L200`, which is very close to line 198. This suggests the first `__deepcopy__` method found is the one I need to modify.

Let's examine the `__deepcopy__` method at line 198. I'll use `sed` to print the lines around line 198 to understand its current implementation.

```bash
sed -n '190,210p' django/forms/fields.py
```

THOUGHT: I've found the `__deepcopy__` method for the `Field` class. As suspected, it performs a shallow copy of `self` and then deep copies the `widget` and copies the `validators` list. However, it *does not* deep copy `self.error_messages`. This confirms the issue described in the PR.

To fix this, I need to add a line to deep copy `self.error_messages` within this `__deepcopy__` method.

The change should look like this:

```python
    def __deepcopy__(self, memo):
        result = copy.copy(self)
        memo[id(self)] = result
        result.widget = copy.deepcopy(self.widget, memo)
        result.validators = self.validators[:]
        result.error_messages = copy.deepcopy(self.error_messages, memo) # Add this line
        return result
```

I will use `sed` to insert this line. I'll insert it after `result.validators = self.validators[:]`.

```bash
sed -i "/result.validators = self.validators[:]/a\        result.error_messages = copy.deepcopy(self.error_messages, memo)" django/forms/fields.py
```

THOUGHT: I've applied the `sed` command to insert the line that deep copies `error_messages`. Now, I need to verify that the change has been correctly applied to the file. I'll use `sed` again to display the relevant lines of the `__deepcopy__` method.

```bash
sed -n '190,210p' django/forms/fields.py
```

THOUGHT: It seems my previous `sed` command did not apply the change. I made a mistake in the `sed` command. The `a\` command in `sed` appends *after* the matched line. I need to ensure the indentation is correct and that the line is actually inserted.

Let me re-examine the `sed` command and the output. The output shows the file *before* the change. This means the `sed -i` command either failed silently or the pattern didn't match as expected.

The pattern `result.validators = self.validators[:]` is correct. The issue might be with the `a\` command and the newline character.

Let's try a different approach with `sed` to ensure the insertion. I will try to replace the `return result` line with the new line and then the `return result` line. This ensures the line is added before the return statement.

```bash
sed -i "/result.validators = self.validators[:]/a\        result.error_messages =
copy.deepcopy(self.error_messages, memo)" django/forms/fields.py
```

I will try the same command again, as it might have been a transient issue. If it fails again, I will try a different `sed` approach.

Wait, I see the problem. The output I received *after* running the `sed` command was the *same* as the output *before* the `sed` command. This indicates the `sed` command did not modify the file. I need to be careful and check the output of the `sed` command.

Let me try to replace the `return result` line with the new line and then the `return result` line. This ensures the line is added before the return statement.

```bash
sed -i '/^        return result/i\        result.error_messages =
copy.deepcopy(self.error_messages, memo)' django/forms/fields.py
```

THOUGHT: My apologies, I made a mistake in the previous turn by not providing an action block. I will correct that now.

I previously tried to insert a line using `sed -i "/result.validators = self.validators[:]/a\..."`. This command did not work as expected.

My new plan is to insert the line `result.error_messages = copy.deepcopy(self.error_messages, memo)` right before the `return result` statement in the `__deepcopy__` method. This is a more robust way to ensure the line is added in the correct place with the correct indentation.

```bash
```

> sed -i '/^   return result/i\   result.error_messages =
> copy.deepcopy(self.error_messages, memo)' django/forms/fields.py
> ```
> ---
>
> THOUGHT: I've applied the `sed` command to insert the line that deep copies
> `error_messages`. Now, I need to verify that the change has been correctly applied to
> the file. I'll use `sed` again to display the relevant lines of the `__deepcopy__` method.
>
> ```bash
> sed -n '190,210p' django/forms/fields.py
> ```
>
> ---
>
> THOUGHT: Great! The `sed` command successfully inserted the line
> `result.error_messages = copy.deepcopy(self.error_messages, memo)` into the
> `__deepcopy__` method. The output confirms the change.
>
> The issue described in the PR is now addressed by deep copying the `error_messages`
> dictionary. I have modified the non-test file `django/forms/fields.py` as required.
>
> I believe the task is complete. I will now submit my work.
>
> ```bash
> echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT && git add -A && git diff
> --cached
> ```

Learnings

(Optional, for 10 most challenging instances)